

std::meta::reflect_constant_{array,string}

Document #: P3617R0 [\[Latest\]](#) [\[Status\]](#)
Date: 2025-05-16
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
- [2 Implementation Approach](#)
- [3 Scheduling for C++26](#)
- [4 Proposal](#)
 - [4.1 Wording](#)
 - [4.2 Feature-Test Macro](#)
- [5 Acknowledgements](#)
- [6 References](#)

§ 1 Introduction

One of the Reflection facilities in flight for C++26 is [\[P3491R2\]\(define_static_{string,object,array}\)](#). It provides three functions:

```
template <ranges::input_range R> // only if the value type is char or char8_t
constexpr auto define_static_string(R&& r) -> ranges::range_value_t<R> const*;

template <class T>
constexpr auto define_static_object(T&& v) -> remove_reference_t<T> const*;

template <ranges::input_range R>
constexpr auto define_static_array(R&& r) -> span<ranges::range_value_t<R> const>;
```

These are very useful additions to C++26.

However, there are cases where having a `span<T const>` or `char const*` is insufficient.

Matthias Wippich sent us some examples of such cases. Consider C++20 code such as:

```
template <size_t N>
struct FixedString {
    char data[N] = {};

    constexpr FixedString(char const(&str)[N]) {
        std::ranges::copy(str, str+N, data);
    }
};

template <FixedString S>
struct Test { };
```

This is a widely used pattern for being able to pass string literals as template arguments. However, there is no way to programmatically produce a string to pass in as an argument:

Approach	Result
<code>using A = Test<"foo">;</code>	✓
<code>using B = [: substitute(^^Test, {reflect_constant("foo"sv)}) :];</code>	✗ Error: std::string_view isn't structural, but even if it was, this wouldn't work because you couldn't deduce the size of S.
<code>using C = Test<define_static_string("foo")>;</code>	✗ Error: cannot deduce the size of S
<code>using D = [:substitute(^^Test, {reflect_constant(define_static_string("foo"))}):];</code>	✗ Error: cannot deduce the size of S

The issue here is that `define_static_string` returns a `char const*`, which loses size information. If, instead, we had a lower layer function that returned a reflection of an array, we could easily use that:

Approach	Result
<code>using E = Test<[:reflect_constant_string("foo"):]>;</code>	✓
<code>using F = [:substitute(^^Test, {reflect_constant_string("foo")}):];</code>	✓

Another situation in which this comes up is in dealing with reflections of members. When you want to iterate over your members one-at-a-time, then `define_static_array` coupled with [\[P1306R4\]\(Expansion Statements\)](#) is perfectly sufficient:

```
template <class T>
auto f(T const& var) -> void {
    template for (constexpr auto M : define_static_array(nsdms(^^T))) {
        do_something_with(var.[:M:]);
    }
}
```

However, some situations require *all* the members at once. Such as in my [struct of arrays](#) implementation. `define_static_array` simply returns a span, but if we had a function that returned a reflection of an array, then combining [\[P1061R10\]\(Structured Bindings can introduce a Pack\)](#) with [\[P2686R5\]\(constexpr structured bindings and references to constexpr variables\)](#) lets us do this instead:

Status Quo	Proposed
<pre>template <auto... V> struct replicator_type { template<typename F> constexpr auto operator>>(F body) const -> decltype(auto) { return body.template operator()<V...>(); } }; template <auto... V> replicator_type<V...> replicator = {}; constexpr auto expand_all(std::span<std::meta::info const> r) -> std::meta::info { std::vector<std::meta::info> rv; for (std::meta::info i : r) { rv.push_back(reflect_value(i)); } return substitute(^^replicator, rv); } template <class T> struct SoaVector { // ... // this is a span<info const> static constexpr auto ptr_mems = define_static_array(nsdms(^^Pointers)); };</pre>	<pre>template <class T> struct SoaVector { // ... // reflection of an object of type info // const[N] static constexpr auto ptr_mems = reflect_constant_array(nsdms(^^Pointers)); // ... };</pre>

Status Quo	Proposed
<pre>// ... auto operator[](size_t idx) const -> T { return [: expand_all(ptr_mems) :] >> [this, idx]<auto... M>{ return T{pointers_.[:M:][idx]...}; }; } };</pre>	<pre>auto operator[](size_t idx) const -> T { constexpr auto [...M] = [: ptr_mems :]; return T{pointers_.[:M:][idx]...}; };</pre>

With the reflection of an array object, we can directly splice it and use structured bindings to get the pack of members we need all in one go — without any layers of indirection. It's a much more direct solution.

Having the higher level facilities is very useful, having this additional lower level facility would also be useful.

§ 2 Implementation Approach

The implementation is actually very straightforward. `define_static_array` already has to produce a reflection of an array in order to do its job. So we simply split that step in two:

Status Quo	Proposed
<pre>template <typename T, T... Vs> inline constexpr T __fixed_array[sizeof...(Vs)] {Vs...}; template <ranges::input_range R> constexpr auto define_static_array(R&& r) -> span<ranges::range_value_t<R> const> { using T = ranges::range_value_t<R>; // produce the array auto args = vector<meta::info>{ ^^ranges::range_value_t<R>; }; for (auto&& elem : r) { args.push_back(meta::reflect_constant(elem)); } auto array = substitute(^^__fixed_array, args); // turn the array into a span return span<T const>{ extract<T const*>(array), extent(type_of(array)); }; }</pre>	<pre>template <typename T, T... Vs> inline constexpr T __fixed_array[sizeof...(Vs)] {Vs...}; template <ranges::input_range R> constexpr auto reflect_constant_array(R&& r) -> meta::info { auto args = vector<meta::info>{ ^^ranges::range_value_t<R>; }; for (auto&& elem : r) { args.push_back(meta::reflect_constant(elem)); } return substitute(^^__fixed_array, args); } template <ranges::input_range R> constexpr auto define_static_array(R&& r) -> span<ranges::range_value_t<R> const> { using T = ranges::range_value_t<R>; // produce the array auto array = reflect_constant_array(r); // turn the array into a span return span<T const>{ extract<T const*>(array), extent(type_of(array)); }; }</pre>

The only difference is exposing `std::reflect_constant_array` instead of it being an implementation detail of `std::define_static_array`. And similar for `std::reflect_constant_string`. It's a simple refactoring.

§ 3 Scheduling for C++26

This is a pure library addition and doesn't strictly need to be in C++26. Moreover, it's implementable on top of [\[P2996R12\]](#) today.

However, I suspect it's a useful one that people will do themselves and that will proliferate anyway. It really seems closer to completing/fixing [\[P3491R2\]](#) than being a novel proposal in its own right.

The other advantage of having it in the compiler rather than user-defined is that the implementation can do better than this by allowing the `__fixed_array` objects to overlap (see discussion in other paper).

So I think we should try to do it for C++26. There is minimal wording review overhead, given that (as you can see below), this feature simply moves the [P3491R2] wording somewhere else, rather than introducing a lot of new wording.

§ 4 Proposal

Add the two facilities:

```
namespace std::meta {
    template <ranges::input_range R> // only if the value_type is char or char8_t
    consteval auto reflect_constant_string(R&& r) -> info;

    template <ranges::input_range R>
    consteval auto reflect_constant_array(R&& r) -> info;
}
```

The naming here mirrors `std::meta::reflect_constant`. Since these facilities return a reflection, it makes sense for them to live in `std::meta`, unlike `define_static_{string,array}` — which only use reflection as an implementation detail.

§ 4.1 Wording

The wording is presented as a diff on [P3491R2].

Change 6.7.2 [intro.object] to instead have the results of `reflect_constant_{string,array}` be potentially non-unique:

- 9 An object is a *potentially non-unique object* if it is
- (9.1) — a string literal object ([lex.string]),
 - (9.2) — the backing array of an initializer list ([dcl.init.ref]),
 - (9.3) — the object declared by a call to `std::define_static_string` `std::meta::reflect_constant_string` or `std::define_static_array` `std::meta::reflect_constant_array`, or
 - (9.4) — a subobject thereof.

And likewise in 6.8.4 [basic.compound]:

- ? A pointer value pointing to a potentially non-unique object *O* ([intro.object]) is *associated* with the evaluation of the *string-literal* ([lex.string]), or initializer list ([dcl.init.list]), or a call to either `std::define_static_string` or `std::define_static_array` — `(meta.define.static)` `std::meta::reflect_constant_string` or `std::meta::reflect_constant_array` ([meta.reflection.array]) that resulted in the string literal object or backing array, respectively, that is *O* or of which *O* is a subobject. [*Note 1*: A pointer value obtained by pointer arithmetic ([expr.add]) from a pointer value associated with an evaluation *E* is also associated with *E*. — *end note*]

Add to [meta.syn], by `reflect_constant`:

```
namespace std::meta {

    // [meta.reflection.result], expression result reflection
    template<class T>
    consteval info reflect_value(const T& value);
    template<class T>
    consteval info reflect_object(T& object);
    template<class T>
    consteval info reflect_function(T& fn);
}
```

```

+ // [meta.reflection.array], promoting to runtime storage
+ template <ranges::input_range R>
+   constexpr info reflect_constant_string(R&& r);
+
+ template <ranges::input_range R>
+   constexpr info reflect_constant_array(R&& r);
+ }

```

Move the corresponding wording from [meta.define.static] to the new clause [meta.reflection.array]. The words here are the same as in the other paper — it's just that we're returning a reflection now instead of extracting a value out of it:

```

1 The functions in this clause are useful for promoting compile-time storage into runtime storage.

   template <ranges::input_range R>
   constexpr info reflect_constant_string(R&& r);

2 Let CharT be ranges::range_value_t<R>.

3 Mandates: CharT is one of char, wchar_t, char8_t, char16_t, or char32_t.

4 Let V be the pack of elements of type CharT in r. If r is a string literal, then V does not include the trailing null
terminator of r.

5 Let P be the template parameter object ([temp.param]) of type const CharT[sizeof...(V)+1] initialized with {V...,
CharT()}.

6 Returns: ^^P.

7 [ Note 2: P is a potentially non-unique object ([intro.object]) — end note ]

   template <ranges::input_range R>
   constexpr info reflect_constant_array(R&& r);

8 Let T be ranges::range_value_t<R>.

9 Mandates: T is a structural type ([temp.param]) and constructible_from<T, ranges::range_reference_t<R>> is
true and copy_constructible<T> is true.

10 Let V be the pack of elements of type T constructed from the elements of r.

11 Let P be the template parameter object ([temp.param]) of type const T[sizeof...(V)] initialized with {V...}.

12 Returns: ^^P.

13 [ Note 3: P is a potentially non-unique object ([intro.object]) — end note ]

```

And then simplify the corresponding wording in [meta.define.static]:

```

1 The functions in this clause are useful for promoting compile-time storage into runtime storage.

   template <ranges::input_range R>
   constexpr const ranges::range_value_t<R>* define_static_string(R&& r);

2 Let CharT be ranges::range_value_t<R>.

3 Mandates: CharT is one of char, wchar_t, char8_t, char16_t, or char32_t.

4 Let V be the pack of elements of type CharT in r. If r is a string literal, then V does not include the trailing null
terminator of r.

5 Let P be the template parameter object ([temp.param]) of type const CharT[sizeof...(V)+1] initialized with {V...,
CharT()}.

```

6 ~~Returns: P .~~

7 ~~[Note 4: P is a potentially non-unique object ([intro.object]) — end note]~~

8 Effects: Equivalent to: `return extract<const ranges::range_value_t<R>*>`
`(meta::reflect_constant_string(r));`

`template <class T>`
`constexpr const remove_cvref_t<T>* define_static_object(T&& t);`

9 Let U be `remove_cvref_t<T>`.

10 Mandates: U is a structural type ([temp.param]) and `constructible_from<U, T>` is `true`.

11 Let P be the template parameter object ([temp.param]) of type `const U` initialized with t .

12 Returns: `std::addressof(P)`.

`template <ranges::input_range R>`
`constexpr span<const ranges::range_value_t<R>> define_static_array(R&& r);`

13 ~~Let T be `ranges::range_value_t<R>`.~~

14 ~~Mandates: T is a structural type ([temp.param]) and `constructible_from<T, ranges::range_reference_t<R>>` is `true` and `copy_constructible<T>` is `true`.~~

15 ~~Let V be the pack of elements of type T constructed from the elements of r .~~

16 ~~Let P be the template parameter object ([temp.param]) of type `const T[sizeof...(V)]` initialized with $\{V, \dots\}$.~~

17 ~~Returns: `span<const T>(P)`.~~

18 ~~[Note 5: P is a potentially non-unique object ([intro.object]) — end note]~~

19 Effects: Equivalent to:

`using T = ranges::range_value_t<R>;`
`meta::info array = meta::reflect_constant_array(r);`
`return span<const T>(extract<const T*>(array), extent(type_of(array)));`

§ 4.2 Feature-Test Macro

Bump the value of `__cpp_lib_define_static` in 17.3.2 [version.syn]:

```
- #define cpp_lib_define_static 2025XX // freestanding, also in <meta>
+ #define __cpp_lib_define_static 2025XX // freestanding, also in <meta>
```

§ 5 Acknowledgements

Thanks to Matthias Wippich for the insightful observation that led to this paper.

§ 6 References

[P1061R10] Barry Revzin, Jonathan Wakely. 2024-11-24. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r10>

[P1306R4] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde, Barry Revzin. 2025-04-24. Expansion Statements.

<https://wg21.link/p1306r4>

[P2686R5] Corentin Jabot, Brian Bi. 2024-11-12. `constexpr` structured bindings and references to `constexpr` variables.

<https://wg21.link/p2686r5>

[P2996R12] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2025-05-11. Reflection for C++26.

<https://wg21.link/p2996r12>

[P3491R2] Barry Revzin, Wyatt Childers, Peter Dimov, Daveed Vandevoorde. 2025-03-14. `define_static_{string,object,array}`.

<https://wg21.link/p3491r2>